Optimization of an Ant Colony Simulation

Data layout reorganization, OpenMP parallelization, and MPI parallelization

Objective: reduce total compute time while preserving the simulation model and benchmark fairness.

SoA

OpenMP

MPI

AMD Ryzen 5 9600X
Windows, g++ 14.2.0
Headless benchmark

Problem and Benchmark Setup

Benchmark conditions

- Terrain: 513 x 513 fractal grid, with nest and food source.
- Reference workload: 5000 ants, headless execution, identical rules for all methods.
- Dominant phase: ant movement, due to random choices and irregular memory access.
- Correctness fixes before measurement: seed initialization and pheromone buffer refresh.

Baseline

0.904 ms/iter

Sequential AoS reference

Bottleneck

0.634 ms/iter

Ant movement dominates the runtime

Logic

Change -> code -> result

Same structure for the three methods

Method 1

Data Layout Reorganization

What changed

- AoS became SoA: position, state, and seed were separated into arrays.
- Movement uses linear indices for terrain and pheromone arrays.
- Touched cells are collected first, then updated once in the buffer.

Representative code

```
vector<uint32_t> m_land_index, m_pher_index;
vector<uint8_t>  m_loaded;
...
for (size_t idx : m_touched_indices)
    buffer[idx] = compute_mark(map, idx);
```

Total

0.874 ms/iter

vs 0.904 ms/iter

Speedup

1.03x

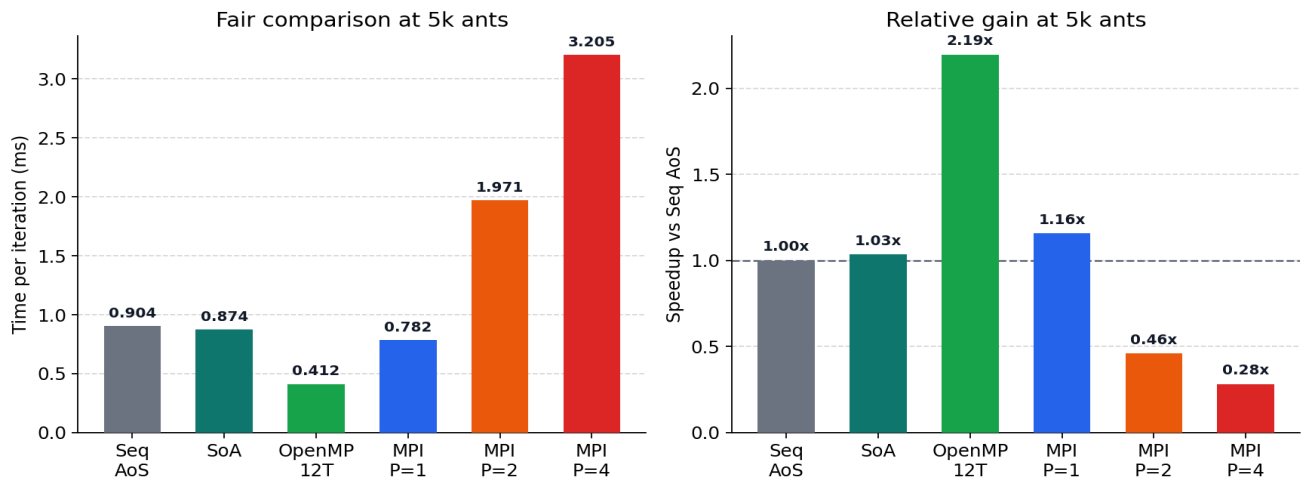
Small but measurable

Reading

Useful

But not transformative

Interpretation: the layout became cleaner, but the movement kernel remained irregular and only weakly vectorizable.



Method 2

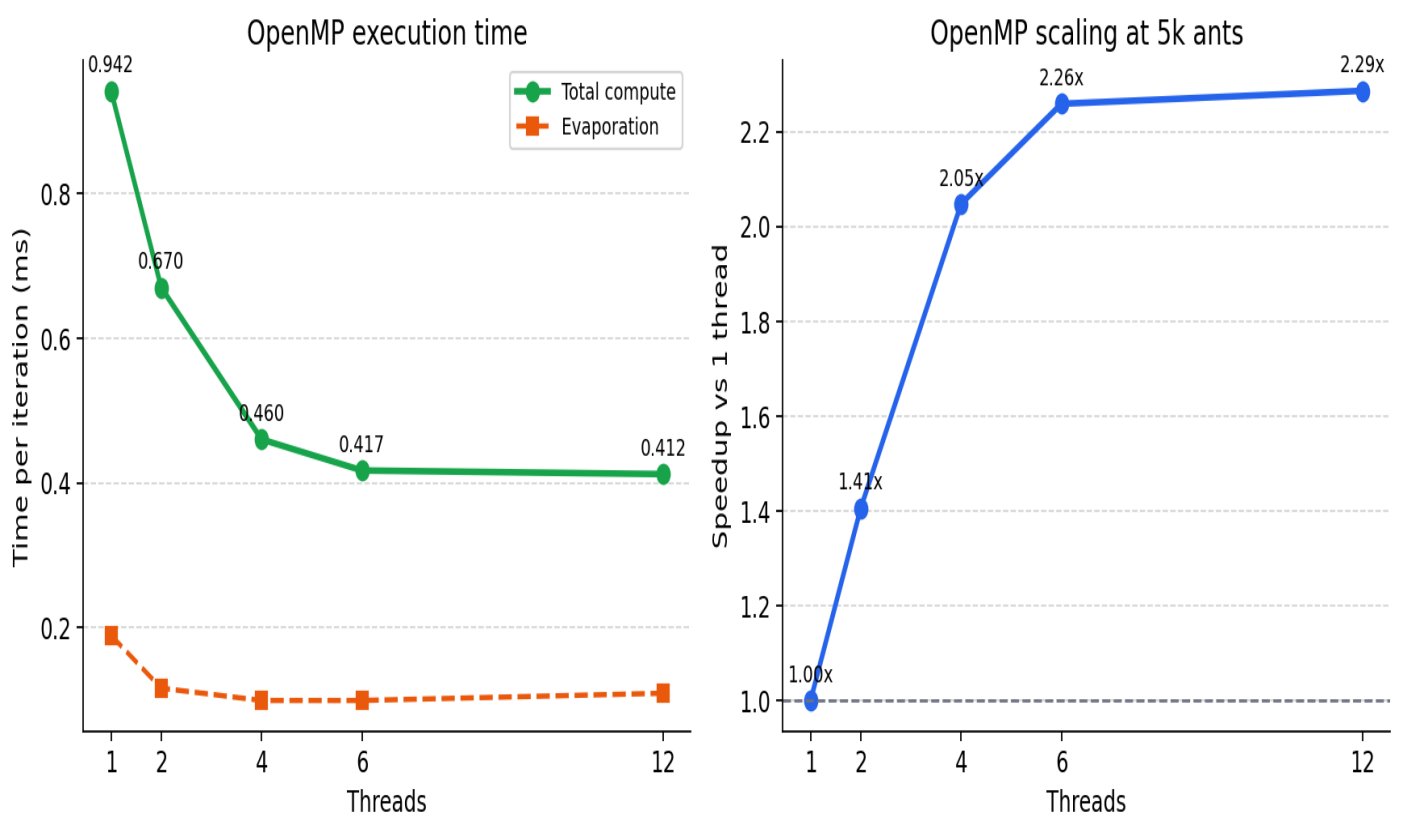
OpenMP Parallelization

What changed

- The final version parallelized ant movement, not only evaporation.
- Thread-local mark buffers removed races in the pheromone update.
- The merge step remained deterministic.

Representative code

```
#pragma omp parallel reduction(+ : food_delta)
{
  ants[i].advance_collect(...);
  local_marks.values[idx] =
phen.compute_mark(pos);
}
merge thread-local marks into the global buffer;
```



Best result: 0.412 ms/iter, corresponding to about 2.29x speedup.

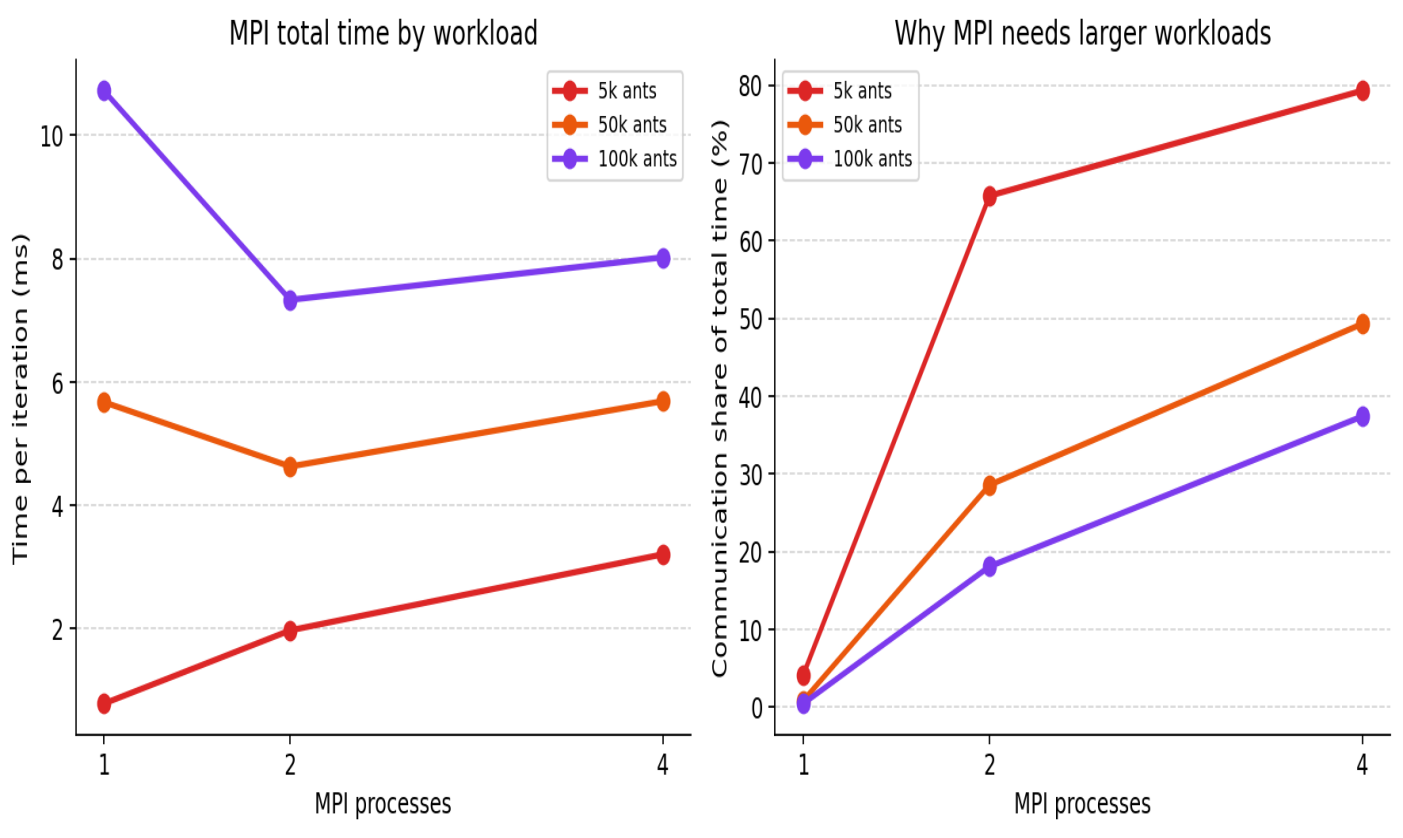
MPI Parallelization

What changed

- Each process owns only a subset of ants, but keeps the full map.
- The pheromone buffer is synchronized with MPI_Allreduce.
- Evaporation is distributed by stripes, followed by a second synchronization.

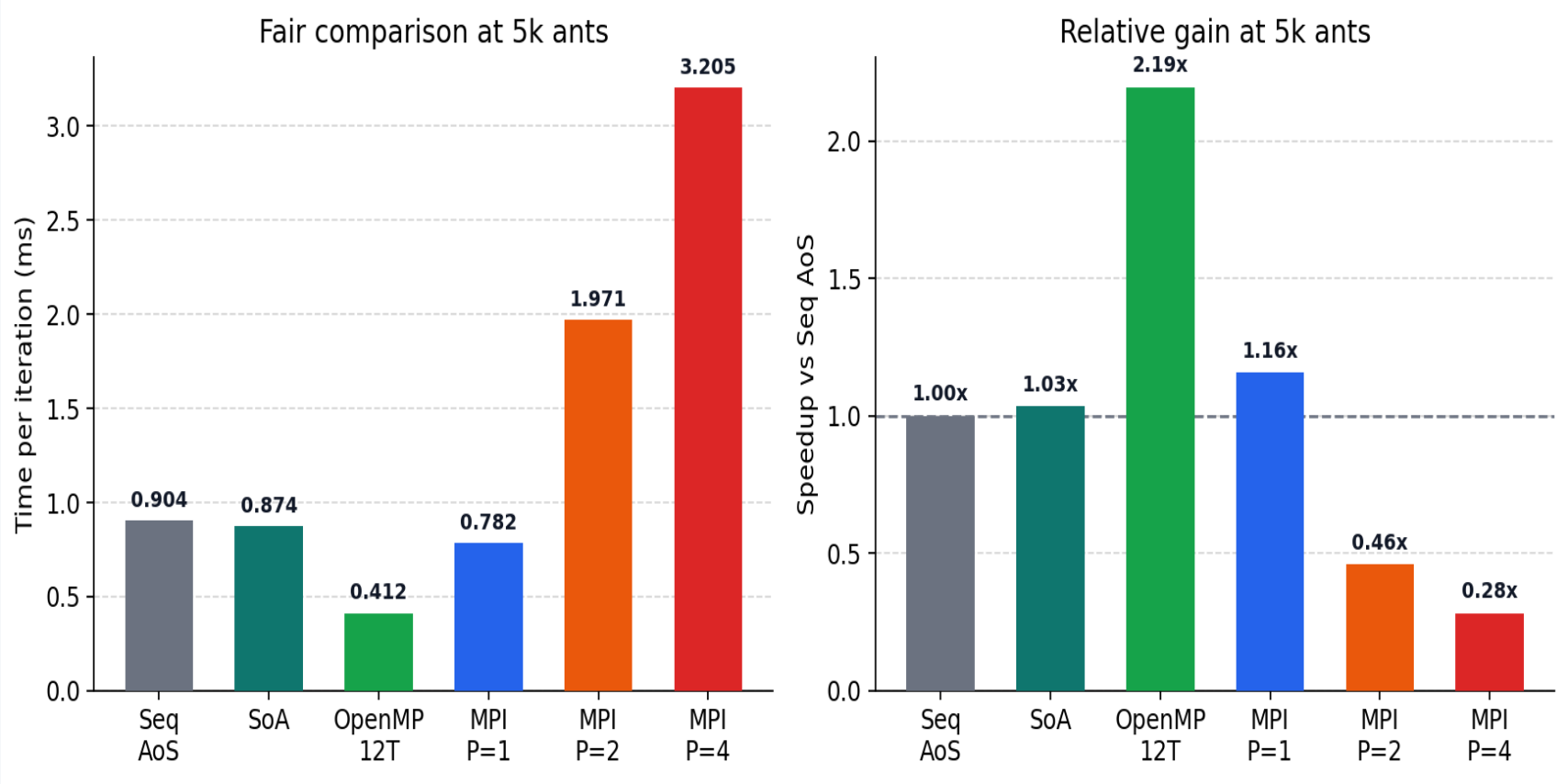
Representative code

```
for (...) {
  for (auto& a : ants) a.advance(...);
  MPI_Allreduce(..., MPI_MAX, ...);
  phen.do_evaporation_stripe(...);
  MPI_Allreduce(..., MPI_MIN, ...);
}
```



Implemented: MPI approach 1. Approach 2 is discussed as a strategy in the report; the bonus was not pursued.

Synthesis in the Common Workload



SoA

1.03x

Small gain

OpenMP

2.29x

Best practical result

MPI

< 1x at 5k

Needs larger workloads

OpenMP is the most effective optimization in the normal regime, while MPI only becomes attractive when the workload is much larger.

Conclusion

Main conclusions

- The first requirement was correctness: seed initialization and pheromone buffer refresh had to be fixed before any serious comparison.
- SoA improved the code organization and produced a small gain, but it did not change the fundamental behavior of the movement kernel.
- OpenMP produced the strongest result because it directly targeted the dominant phase of the simulation.
- MPI approach 1 was implemented and measured. MPI approach 2 was covered as a proposed strategy, which satisfies the non-bonus requirement of the assignment.

OpenMP gave the best balance between implementation effort and performance gain.